Introdução à Orientação a Objetos

Aula 12 — Interfaces

Programação Orientada a Objetos

Semana 12

Sumário

1. Abertura
2. Etapa 1: o contrato `Produto`
3. Etapa 2: várias interfaces
4. Etapa 3: interface $\times$ classe abstrata
5. Etapa 4: `default`, `static` e `@FunctionalInterface`
6. Fechamento

Onde paramos

Na aula 11 a hierarquia da livraria ficou assim:

- `Livro` é **abstrata**; `Ebook`, `LivroFisico` e `Audiolivro` herdam dela
- cada filha sobrescreve o que precisa (`toString`, `aplicaDescontoDe`, `getFormato`)

E deixamos uma pergunta no ar:

`Ebook` e `Audiolivro` podem ser **baixados**; `LivroFisico` não.

Um método `baixar()` em `Livro` seria uma **mentira** para o livro físico.

E se a loja passar a vender **revistas**? Elas nem são livros.

Hoje

Quatro perguntas, uma etapa para cada uma:

1. Como tratar `Livro` e `Revista` pelo mesmo tipo, **sem** parentesco? *contrato*
2. Como dar uma capacidade (desconto) só a **alguns** produtos? *várias interfaces*
3. Quando usar **interface** e quando usar classe abstrata? *comparação*
4. Como **evoluir** uma interface sem quebrar quem a usa? `default`

Em cada etapa: um pouco de conceito, e depois **em duplas**, no código da livraria.

Como vamos trabalhar

Cada etapa segue o mesmo ciclo da aula 11:

1. o professor apresenta o **conceito** (10 a 15 min)
2. a dupla **implementa** — quem digita troca a cada etapa
3. **checkpoint**: rode o teste e compare com a saída esperada
4. saída bateu? siga para a próxima etapa

- Travou? Tente por 5 minutos, releia as pistas, depois chame o professor
- O enunciado completo está na lista (PDF), com os mesmos testes e saídas

Mapa da aula

Etapa	O que fazemos	Tempo
1	Interface `Produto` e a classe `Revista`	45 min
2	Interface `Promocional` e várias interfaces	45 min
3	Interface `Baixavel`: interface $\times$ classe abstrata	45 min
4	`default`, `static` e `@FunctionalInterface`	45 min
	Fechamento	20 min

O projeto de partida

Baixe o `.zip` no Moodle e abra a pasta `src` na IDE. É o projeto da aula 11:

Pacote	Classes
`br.edu.ifsp.livraria`	`Autor`, `Editora`, `CarrinhoDeCompras`, `Catalogo`
`br.edu.ifsp.livraria.produtos`	`Livro`, `Ebook`, `LivroFisico`, `Audiolivro`
`br.edu.ifsp.livraria.testes`	`CadastroDeLivros` (o `main`), `TiposDeLivro`

O `main` de `CadastroDeLivros` deve imprimir:

```
Livros cadastrados: 2
Total do carrinho: 148.9
```

O `CarrinhoDeCompras` está enxuto: só `adiciona`, `getQuantidade`, `getTotal` e `getLivros`.

Chegou a revista

A livraria agora vende `Revista`, que tem nome, valor e editora. O carrinho só aceita `Livro`:

```
1 public class CarrinhoDeCompras {
2     private List<Livro> livros = new ArrayList<>();
3
4     public void adiciona(Livro livro) { ... }
5     // e a Revista?
6 }
```

Três saídas, e **todas** têm custo:

1. **duplicar** o método: `adiciona(Revista)`. Código repetido e um método novo a cada produto
2. `Revista extends Livro`: a revista ganha **autor** e **ISBN** que não fazem sentido
3. criar uma superclasse `Produto`: funciona, mas aumenta o **acoplamento**

O problema da herança

- **Acoplamento:** o quanto uma classe depende de outra
- A herança cria a relação mais forte possível: mudar a superclasse influencia **todas** as filhas, e as filhas das filhas
- Se só algumas filhas precisam da mudança, as outras são obrigadas a sobrescrever só para "desligar" o comportamento
- Uma classe só pode ter **uma** superclasse

Queremos o polimorfismo **sem** amarrar as classes numa árvore.

Interface: um contrato

Em Java, o contrato se chama **interface**. Só diz **o que** fazer, não **como**:

```
1 public interface Produto {
2     String getNome();
3
4     double getValor();
5 }
```

- Palavra reservada `interface` no lugar de `class`
- Métodos **sem corpo**, e já são `public abstract` (não precisa escrever)
- **Não é** interface gráfica (GUI) e **não é** a "interface da classe" (seus métodos públicos): é um recurso novo

Assinando o contrato

A classe **assina** o contrato com `implements` e é obrigada a implementar todos os métodos:

```
1 public abstract class Livro implements Produto {
2     // getNome() e getValor() ja existem
3 }
4
5 public class Revista implements Produto {
6     private String nome;
7     private double valor;
8     // getNome() e getValor() ...
9 }
```

Esqueceu um método? Não compila:

```
error: Revista is not abstract and does not override abstract method getNome() in
    Produto
```

Interface como tipo

Quem assina o contrato pode ser **referenciado** pelo tipo da interface: é polimorfismo, sem herança.

```
1 public void adiciona(Produto produto) {
2     this.produtos.add(produto);
3 }
4
5 Produto p1 = new LivroFisico(autor);   // ok
6 Produto p2 = new Revista("Java Magazine", 20.00);   // ok
7 Produto p3 = new Produto();   // erro
```

```
error: Produto is abstract; cannot be instantiated
```

- Pela referência `Produto` só se chamam os métodos **do contrato**
- Um **único** `adiciona(Produto)` serve a livros, revistas e a qualquer produto futuro

Etapa 1: objetivo

1. Crie a interface `Produto` (pacote `produtos`) com `String getNome()` e `double getValor()`
2. `Livro` passa a `implements Produto` (marque os dois *getters* com `@Override`)
3. Crie `Revista implements Produto`: atributos `nome` e `valor`, construtor `Revista(String nome, double valor)` e os dois *getters*
4. `CarrinhoDeCompras` passa a guardar uma `List<Produto>`:
   - `adiciona(Produto produto)`
   - `getProdutos()` no lugar de `getLivros()`
   - `getTotal()` e `getQuantidade()` continuam

Etapa 1: pistas

- Comece pela interface: o compilador vai apontar tudo que falta
- `Livro` já tem `getNome()` e `getValor()`: só some o `implements`
- Não precisa mexer em `Ebook`, `LivroFisico` e `Audiolivro`: herdam o contrato
- Troque `Livro` por `Produto` em **todo** o `CarrinhoDeCompras`
- Experimente apagar o `getNome()` da `Revista` e ler a mensagem do compilador; depois devolva

Etapa 1: checkpoint (1/2)

Acrescente ao final do `main`:

```
System.out.println("--- Etapa 1 ---");
Autor aniche = new Autor("Mauricio Aniche");

Ebook ebook = new Ebook(aniche);
ebook.setNome("Test-Driven Development");
ebook.setIsbn("978-85-5519-024-8");
ebook.setValor(30.00);
ebook.setWaterMark("aluno@ifsp.edu.br");

LivroFisico fisico = new LivroFisico(aniche);
fisico.setNome("Test-Driven Development");
fisico.setIsbn("978-85-5519-023-1");
fisico.setValor(60.00);

Audiolivro audio = new Audiolivro(aniche, 395);
audio.setNome("Test-Driven Development");
audio.setIsbn("978-85-5519-025-5");
audio.setValor(40.00);
```

Etapa 1: checkpoint (2/2)

```
Revista revista = new Revista("Java Magazine", 20.00);

CarrinhoDeCompras carrinho = new CarrinhoDeCompras();
carrinho.adiciona(ebook);
carrinho.adiciona(fisico);
carrinho.adiciona(audio);
carrinho.adiciona(revista);

System.out.println(carrinho.getQuantidade());
System.out.println(carrinho.getTotal());
Produto produto = revista;
System.out.println(produto.getNome());
for (Produto item : carrinho.getProdutos()) {
    System.out.println(item.getNome() + ": "
            + String.format("R$ %.2f", item.getValor()));
}
```

Etapa 1: saída esperada

```
--- Etapa 1 ---
4
150.0
Java Magazine
TEST-DRIVEN DEVELOPMENT: R$ 30,00
TEST-DRIVEN DEVELOPMENT: R$ 60,00
TEST-DRIVEN DEVELOPMENT: R$ 40,00
Java Magazine: R$ 20,00
```

- Os nomes dos livros saem em maiúsculas: o `setNome` da `Livro` faz isso
- Os valores saem com vírgula se a JVM estiver em português do Brasil

O desconto não vale para todo produto

- Hoje o `aplicaDescontoDe` está na `Livro`: a `Revista` não o tem, e o `Audiolivro` o herda sem ter pedido
- Colocar o método em `Produto` obriga **todo** produto a ter desconto
- Pior: se um dia existir um produto sem desconto, ele terá de implementar um método só para devolver `false`

Solução: outra interface, **pequena**, só com essa capacidade:

```
1 public interface Promocional {
2     boolean aplicaDescontoDe(double porcentagem);
3 }
```

Só implementa quem tem desconto. Quem não assina, não tem a obrigação.

Uma classe, várias interfaces

Assim como se assinam vários contratos na vida, uma classe pode implementar **várias** interfaces, separadas por vírgula:

```
1 public class Revista implements Produto, Promocional { ... }
2
3 public class LivroFisico extends Livro implements Promocional { ... }
4
5 public class Ebook extends Livro implements Promocional { ... }
6
7 public class Audiolivro extends Livro { ... }   // sem desconto
```

- Com herança e interface juntas: primeiro `extends`, depois `implements`
- Herança: **uma** superclasse. Interfaces: quantas forem preciso

Quem assina o quê (etapa 2)

Classe	`Produto`	`Promocional`	Limite de desconto
`LivroFisico`	sim (via `Livro`)	sim	30%
`Ebook`	sim (via `Livro`)	sim	15%
`Audiolivro`	sim (via `Livro`)	**não**	—
`Revista`	sim	sim	10%

- Para o `Audiolivro` não ter desconto basta **não assinar** o contrato
- O método sai da `Livro` e vai para as filhas que realmente o têm

Perguntando pela capacidade

O carrinho tem `Produtos`, mas só alguns são `Promocional`. Como aplicar o desconto?

```java
public int aplicaDescontoEmTodos(double porcentagem) {
    int comDesconto = 0;
    for (Produto produto : this.produtos) {
        if (produto instanceof Promocional) {
            Promocional promocional = (Promocional) produto;
            if (promocional.aplicaDescontoDe(porcentagem)) {
                comDesconto++;
            }
        }
    }
    return comDesconto;
}
```

- `instanceof` funciona com interface: "este objeto **sabe** dar desconto?"
- Diferente da aula 11: o teste não é por classe concreta (`LivroFisico`), é por **capacidade**

Etapa 2: objetivo

1. Crie a interface `Promocional` com `boolean aplicaDescontoDe(double porcentagem)`
2. `Livro`: **remova** `aplicaDescontoDe` e a constante `DESCONTO_MAXIMO`
3. `LivroFisico implements Promocional`: constante `DESCONTO_MAXIMO_FISICO = 0.3`; acima disso imprime `Desconto acima do permitido` e devolve `false`
4. `Ebook implements Promocional`: o método agora é **completo** (limite de 15%, sem `super`), com a mesma mensagem de antes
5. `Audiolivro` **não** assina `Promocional`
6. `Revista implements Produto, Promocional`: limite `DESCONTO_MAXIMO_REVISTA = 0.1`; acima disso imprime `Desconto acima do permitido para revista`
7. `CarrinhoDeCompras.aplicaDescontoEmTodos(double)`: devolve quantos produtos receberam o desconto

Etapa 2: pistas

- O desconto reduz o valor: `setValor(getValor() - getValor() * porcentagem)`
- O `Ebook` não chama mais `super.aplicaDescontoDe(...)`, porque o método não existe mais na `Livro`
- Marque os métodos com `@Override`: agora eles vêm do contrato
- No carrinho, teste com `instanceof Promocional` e só então converta
- O `TiposDeLivro` continua compilando: o `Ebook` ainda tem `aplicaDescontoDe`

Etapa 2: checkpoint

Acrescente ao final do `main` (reaproveita as variáveis da etapa 1):

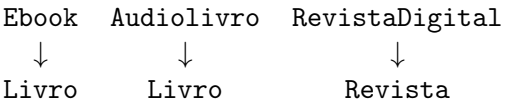
```
System.out.println("--- Etapa 2 ---");
System.out.println(carrinho.aplicaDescontoEmTodos(0.1));
System.out.println(carrinho.getTotal());
Promocional promocional = revista;
System.out.println(promocional.aplicaDescontoDe(0.5));
System.out.println(revista instanceof Promocional);
System.out.println(audio instanceof Promocional);
```

```
--- Etapa 2 ---
3
139.0
Desconto acima do permitido para revista
false
true
false
```

- Três receberam 10%: e-book, livro físico e revista; o audiolivro ficou de fora

Quem pode ser baixado?

- `Ebook` e `Audiolivro`: sim. `LivroFisico`: não
- A loja passa a vender a `RevistaDigital`, que também pode ser baixada



O que une os três **não** está na árvore de herança. Nenhuma superclasse resolve:

- `baixar()` em `Livro`: o livro físico mente, e a revista digital fica de fora
- Uma superclasse comum a livros e revistas: tudo de novo

A interface Baixavel

```java
public interface Baixavel {
    String baixar();
}

public class Ebook extends Livro implements Promocional, Baixavel { ... }
public class Audiolivro extends Livro implements Baixavel { ... }
public class RevistaDigital extends Revista implements Baixavel { ... }
```

- Classes **sem parentesco** compartilham um tipo: `List<Baixavel>`
- `RevistaDigital` herda o desconto de `Revista` e assina o novo contrato
- Quem chama `baixar()` não precisa saber **qual** classe está do outro lado

Regras da interface

- Os métodos são `public`; a classe que os implementa **deve** declará-los `public`

```
1 public class Ebook implements Baixavel {
2     String baixar() { ... }   // sem public
3 }
```

```
error: baixar() in Ebook cannot implement baixar() in Baixavel
  attempting to assign weaker access privileges; was public
```

- Variáveis de interface são implicitamente `public static final`: são constantes
- Classe que assina e **não** implementa tudo precisa ser `abstract`
- Uma interface não tem construtor, nem atributos de instância

Classe abstrata × interface

	Classe abstrata	**Interface**
Palavra-chave	`abstract class`	`interface`
Quantas uma classe usa	**uma** (`extends`)	**várias** (`implements`)
Atributos de instância	pode ter	não pode
Construtor	pode ter	não tem
Métodos com corpo	pode ter	`default`, `static` e `private` (Java 9+)
Relação	"**é um**" (`Ebook` é um `Livro`)	"**sabe fazer**" (`Ebook` é `Baixavel`)

- **Classe abstrata:** base comum, com estado e código compartilhado (`Livro`)
- **Interface:** capacidade que classes de famílias diferentes podem ter (`Promocional`, `Baixavel`)

Etapa 3: objetivo

1. Crie a interface `Baixavel` com `String baixar()`
2. `Ebook implements` também `Baixavel`: `"Baixando "+ nome + "com marca d'agua "+ waterMark`
3. `Audiolivro implements Baixavel`: `"Baixando "+ nome + "(6h35min de audio)"`, com a duração formatada
4. Crie `RevistaDigital extends Revista implements Baixavel`: construtor `(String nome, double valor)` e `"Baixando "+ nome + "em PDF"`
5. `CarrinhoDeCompras.getBaixaveis()`: devolve uma `List<Baixavel>` com os produtos do carrinho que podem ser baixados

O `LivroFisico` **não** assina `Baixavel`.

Etapa 3: pistas

- O texto do `Audiolivro` usa o método `getDuracaoFormatada()` que já existe
- `RevistaDigital` usa `super(nome, valor)` e herda `getNome()`, `getValor()` e o desconto
- `getBaixaveis()` segue o modelo do `aplicaDescontoEmTodos`: `instanceof Baixavel`, *cast*, `add`
- Repare que o `Ebook` agora lista `Promocional, Baixavel` depois do `extends Livro`
- Experimente tirar o `public` do `baixar()` do `Ebook` e ler o erro; depois devolva

Etapa 3: checkpoint

Acrescente ao final do `main`:

```
System.out.println("--- Etapa 3 ---");
RevistaDigital digital = new RevistaDigital("Programar", 15.00);
carrinho.adiciona(digital);
for (Baixavel baixavel : carrinho.getBaixaveis()) {
    System.out.println(baixavel.baixar());
}
```

```
--- Etapa 3 ---
Baixando TEST-DRIVEN DEVELOPMENT com marca d'agua aluno@ifsp.edu.br
Baixando TEST-DRIVEN DEVELOPMENT (6h35min de audio)
Baixando Programar em PDF
```

- O livro físico e a revista comum não aparecem: não são `Baixavel`

Evoluindo uma interface

O Java 8 permitiu que a interface tenha métodos **com corpo**. Por quê?

- Se acrescentarmos um método abstrato a `Promocional`, **todas** as classes que a assinam deixam de compilar
- Em interfaces muito usadas (as da API do Java), isso quebraria milhões de programas

Solução: método `default`, com implementação padrão:

```
1 public interface Promocional {
2     boolean aplicaDescontoDe(double porcentagem);
3
4     default boolean aplicaDescontoDe10Porcento() {
5         return aplicaDescontoDe(0.1);
6     }
7 }
```

Regras do default

```
1 Revista revista = new Revista("Java Magazine", 20.00);
2 System.out.println(revista.aplicaDescontoDe10Porcento());   // true
```

- Todas as classes que assinam `Promocional` ganham o método **sem alterar uma linha**
- A classe pode **sobrescrever** o `default` se quiser outro comportamento
- O `default` só chama outros métodos da interface: **não acessa atributos**, porque a interface não tem estado
- Não é herança múltipla: foi criado para **evoluir** interfaces, não para compartilhar código nem estado

Método `static` na interface

Também com corpo, mas pertence à **interface**, não aos objetos. Chama-se pelo nome da interface:

```
1 public interface Promocional {
2     boolean aplicaDescontoDe(double porcentagem);
3
4     static boolean descontoValido(double porcentagem) {
5         return porcentagem > 0 && porcentagem < 1;
6     }
7 }
8
9 System.out.println(Promocional.descontoValido(1.5));   // false
```

- Serve para utilitários ligados ao contrato
- Não é herdado: `revista.descontoValido(0.1)` **não** compila

Método `private` na interface (Java 9)

Dois `default` repetem a mesma validação? Extraia para um método `private`:

```
1  public interface Descontavel {
2      boolean aplicaDescontoDe(double porcentagem);
3      default boolean aplicaDescontoDe5Porcento() {
4          return aplicaSeValido(0.05);
5      }
6      default boolean aplicaDescontoDe10Porcento() {
7          return aplicaSeValido(0.1);
8      }
9      private boolean aplicaSeValido(double porcentagem) {
10         return porcentagem > 0 && porcentagem < 1 && aplicaDescontoDe(porcentagem);
11     }
12 }
```

- Só existe **dentro** da interface: quem a implementa não o vê nem o herda
- Serve para dividir código entre os `default`; **não** vira exercício

Interface funcional

Uma interface com **um único método abstrato** é chamada de interface funcional. `Produto` tem dois; `Promocional` e `Baixavel` têm um cada.

```
1 @FunctionalInterface
2 public interface Promocional {
3     boolean aplicaDescontoDe(double porcentagem);
4     boolean naoSouMaisFuncional();
5 }
```

```
error: Unexpected @FunctionalInterface annotation
  Promocional is not a functional interface
    multiple non-overriding abstract methods found in interface Promocional
```

- A anotação é opcional: o compilador confere que a interface **continua** com um só método abstrato
- Métodos `default` e `static` **não** contam; a base das expressões *lambda* (adiante)

Etapa 4: objetivo

1. Em `Promocional`, acrescente o método `default boolean aplicaDescontoDe10Porcento()`, que chama `aplicaDescontoDe(0.1)`
2. Acrescente o método `static boolean descontoValido(double porcentagem)`: verdadeiro se for maior que 0 e menor que 1
3. Marque `Promocional` com `@FunctionalInterface`
4. **Experimento:** acrescente um segundo método abstrato a `Promocional`, leia o erro e desfaça

Nenhuma classe (`Ebook`, `LivroFisico`, `Revista`, `RevistaDigital`) precisa de alteração.

Etapa 4: pistas

- Dentro do `default`, chame `aplicaDescontoDe(0.1)` direto, sem `this`: o método abstrato é da própria interface
- O `static` se chama com o **nome da interface**: `Promocional.descontoValido(...)`
- Se alguma classe parar de compilar, o `default` ou o `static` foi escrito fora da interface
- No experimento, não esqueça de remover o segundo método abstrato antes do checkpoint

Etapa 4: checkpoint

Acrescente ao final do `main`:

```
System.out.println("--- Etapa 4 ---");
Promocional p = digital;
System.out.println(p.aplicaDescontoDe10Porcento());
System.out.println(digital.getValor());
System.out.println(Promocional.descontoValido(0.1));
System.out.println(Promocional.descontoValido(1.5));
```

```
--- Etapa 4 ---
true
13.5
true
false
```

- A saída de **todas** as etapas anteriores continua a mesma

Desafio bônus: interface que estende interface

Uma interface pode **estender** outras, inclusive várias:

```
1 public interface ProdutoPromocional extends Produto, Promocional {
2 }
```

- Quem implementa `ProdutoPromocional` precisa implementar os métodos de **ambas**
- Troque o `implements Produto, Promocional` da `Revista` por `implements ProdutoPromocional`

```
1 System.out.println("--- Bonus ---");
2 ProdutoPromocional pp = digital;
3 System.out.println(pp.getNome());
4 System.out.println(pp.aplicaDescontoDe(0.05));
```

```
--- Bonus ---
Programar
true
```

Exit ticket

Em uma folha, com o nome da dupla:

1. Por que a `Revista` não herdou de `Livro`? O que ela ganhou ao assinar `Produto`?
2. Por que o desconto ficou numa interface separada (`Promocional`) em vez de ir para `Produto`?
3. Cite duas diferenças entre classe abstrata e interface e diga quando usar cada uma.
4. Para que serve um método `default`? Por que ele não transforma a interface em herança múltipla?

Resumo

- **Etapa 1:** `interface` é um contrato; `implements`; a interface serve de tipo, sem herança
- **Etapa 2:** uma classe implementa **várias** interfaces; interfaces pequenas dão capacidades só a quem precisa
- **Etapa 3:** classes sem parentesco compartilham uma interface; métodos `public`; abstrata $\times$ interface
- **Etapa 4:** `default` e `static`; interface funcional e `@FunctionalInterface`

Entrega: o projeto em `.zip` no Moodle, no prazo indicado.

Próxima aula

- Hoje usamos `instanceof` e *cast* no `aplicaDescontoEmTodos` e no `getBaixaveis`
- Será que dá para fazer sem perguntar o tipo? Quando o *cast* é inevitável e quando é sinal de projeto ruim?
- Na semana 13: **polimorfismo**, aprofundado, com herança e interfaces juntas

Curiosidade (Java 16 ou mais novo): o *cast* pode sumir com `instanceof` de padrão

```
1 if (produto instanceof Promocional promocional) {   // sem a linha do cast
2     promocional.aplicaDescontoDe(porcentagem);
3 }
```

Referências

SCHILDT, Herbert. **Java**: a beginner's guide. 9. ed. Nova York: McGraw Hill, 2022. (cap. 8, seção *Interfaces*)
TURINI, Rodrigo. **Desbravando Java e Orientação a Objetos**: um guia para o iniciante da linguagem. [S.l.: s.n.], [20–]. (seções 7.1 a 7.3)
Complementares (documentação oficial): ORACLE. **The Java® Language Specification**: Java SE 17 Edition. Cap. 9, *Interfaces*. Disponível em: https://docs.oracle.com/javase/specs/jls/se17/html/jls-9.html. Acesso em: 30 out. 2026.
ORACLE. **The Java™ Tutorials**: Defining an Interface. Disponível em: https://docs.oracle.com/javase/tutorial/java/IandI/interfaceDef.html. Acesso em: 30 out. 2026.
DEV.JAVA. **Interfaces**: defining interfaces. Disponível em: https://dev.java/learn/interfaces/defining-interfaces/. Acesso em: 30 out. 2026.
ORACLE. **Java Language Changes Summary**. Disponível em: https://docs.oracle.com/en/java/javase/25/language/java-language-changes-summary.html. Acesso em: 30 out. 2026.